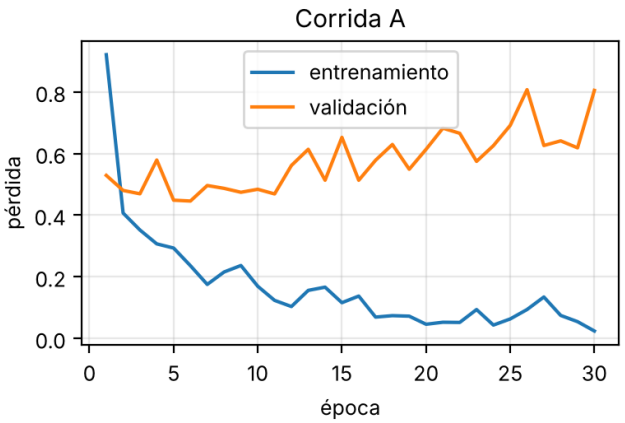
SI7011 · Evento evaluativo 1 · Parte A: leer la evidencia

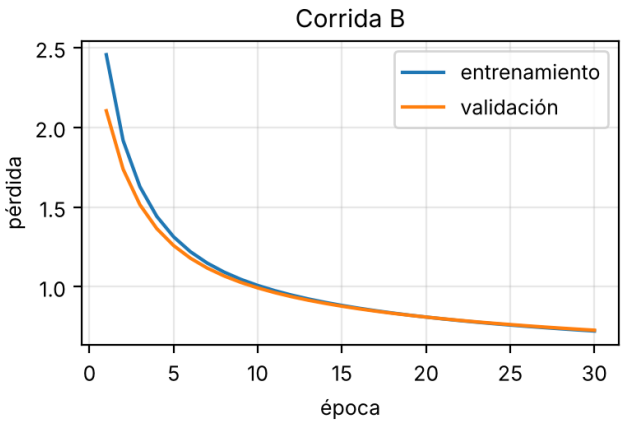
Nombre: _____ **Documento:** _____

Individual · sin computador ni apuntes · 30 minutos · 6 preguntas, 1 punto cada una. Respuestas cortas: una o dos frases por literal. Cuando la pregunta pide evidencia, cita un número leído de la figura.

---

1. Dos corridas





a) Diagnostica cada corrida en una frase, citando lo que ves en las curvas.

Corrida A: Sufre de sobreajuste (*overfitting*) porque la pérdida de entrenamiento baja hasta casi 0.02 en la época 30, mientras que la de validación empeora subiendo desde 0.45 (época 5) hasta 0.80.

Corrida B: Presenta un aprendizaje estable sin sobreajuste (con margen para seguir entrenando), ya que las curvas de entrenamiento y validación bajan pegadas desde ~2.5 y 2.1 hasta llegar juntas a ~0.75 en la época 30.

b) Propón **una** acción para cada corrida y di qué cambio en las curvas confirmaría que funcionó.

Corrida A: Aplicaría regularización (como *Dropout* o *Weight Decay*) o parada temprana (*early stopping*) en la época 5; funcionó si la curva de validación deja de subir hacia 0.80 y se mantiene baja cerca de la curva de entrenamiento.

Corrida B: Aumentaría el número de épocas o subiría un poco la tasa de aprendizaje (η); funcionó si ambas curvas continúan bajando por debajo de 0.75 sin separarse.

2. Un cálculo de varianza

Una capa ReLU con $n_{\text{in}} = 400$ y pesos $W \sim \mathcal{N}(0,\, 0.05^2)$. Supón que ReLU conserva la mitad de la varianza.

a) ¿Por qué factor se multiplica Var(a_l) en cada capa?

Se multiplica por un factor de **0.5** (la varianza se reduce a la mitad en cada capa), ya que $1/2 \cdot n_{in} \cdot \sigma^2 = 0.5 \times 400 \times 0.05^2 = 0.5$.

b) Después de 10 capas, ¿cuánto vale std(a_{10}) en proporción a la inicial?

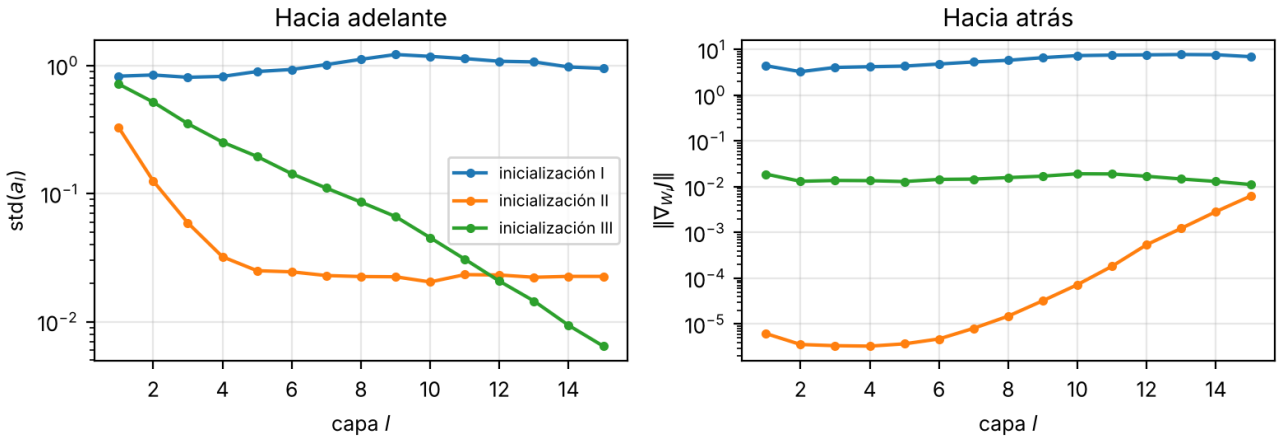
Como la desviación estándar cambia en un factor de $\sqrt{0.5} = 1/\sqrt{2}$ por capa, después de 10 capas vale $(1/\sqrt{2})^{10} = 1/32 \approx$ **0.03125** (es decir, el 3.125 % de la inicial).

c) ¿Qué desviación estándar σ conserva la varianza? ¿Cómo se llama esa inicialización?

Para que el factor sea igual a 1, se necesita $\sigma = \sqrt{(2/400)} \approx$ **0.0707**, y este método se conoce como **inicialización de He** (o *Kaiming*).

3. Tres inicializaciones

Un MLP de 15 capas ocultas con ReLU y 256 unidades, medido **antes de entrenar** sobre un lote, con tres inicializaciones: la de PyTorch por defecto, Xavier y He (en algún orden).



a) Asigna I, II y III a {PyTorch por defecto, Xavier, He}. Justifica cada una con un número de las gráficas.

Inicialización I = He: Compensa exactamente la pérdida de ReLU y mantiene la señal hacia adelante constante alrededor de std(a_l) ≈ 10^0 (1.0) y el gradiente hacia atrás estable cerca de 5 × 10^0 en todas las capas.

Inicialización II = PyTorch por defecto: Sus pesos iniciales son tan pequeños que las activaciones caen rápido a ~2 × 10^{-2} desde la capa 5 y los gradientes se desploman hasta ~4 × 10^{-6} en las primeras capas.

Inicialización III = Xavier: No compensa el corte a la mitad de ReLU, por lo que la señal hacia adelante cae en línea recta (en escala logarítmica) desde ~0.7 en la capa 1 hasta ~6 × 10^{-3} en la capa 15.

b) ¿Cuál entrenarías con SGD? ¿Qué les pasaría a las primeras capas con las otras dos?

Entrenaría la **Inicialización I (He)** porque conserva señales y gradientes saludables en toda la red. Con las otras dos (sobre todo en la II, con gradientes de 10^{-5} a 10^{-6}), las primeras capas sufrirían desvanecimiento del gradiente y se quedarían congeladas sin aprender.

4. La pérdida inicial

Un problema con $C = 7$ clases balanceadas. Tres compañeros reportan la pérdida de **validación antes de entrenar** (entropía cruzada). Interpreta cada valor en una frase: ¿está bien? Si no, ¿qué sospechas?

a) 1.95

Está bien, porque sin entrenar la red asigna una probabilidad al azar de 1/7 a cada clase y la pérdida esperada es $-\ln(1/7) = \ln(7) \approx 1.95$.

b) 38.2

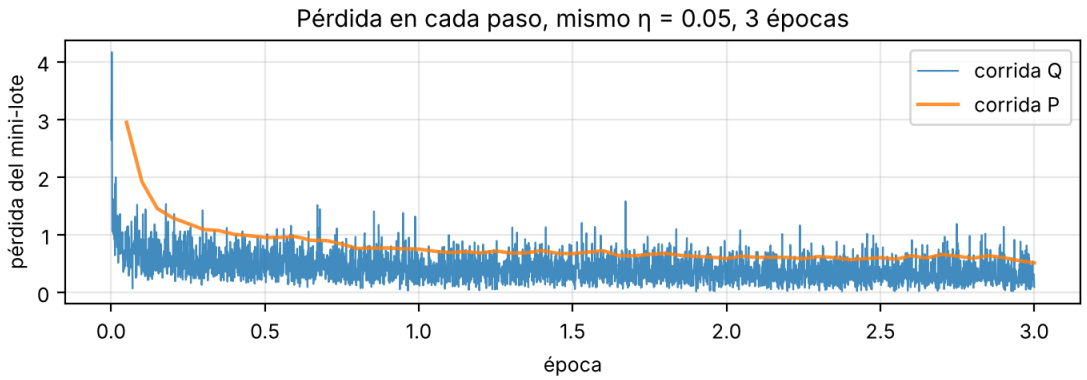
Está mal por ser excesivamente alta; sospecho que los pesos iniciales son demasiado grandes (generando predicciones al azar pero con altísima confianza errónea) o hay un error en el cálculo de la función de pérdida.

c) 0.03

Está mal por ser casi cero sin haber entrenado; sospecho que se evaluó un modelo que ya estaba entrenado o que existe fuga de datos (la etiqueta de respuesta se filtró dentro de las entradas).

5. Tamaño del lote

El mismo modelo con el mismo $\eta = 0.05$ durante 3 épocas. Una corrida usa lotes de 16 y la otra lotes de 1024. Corrida P: 0.14 s por época. Corrida Q: 0.82 s por época.



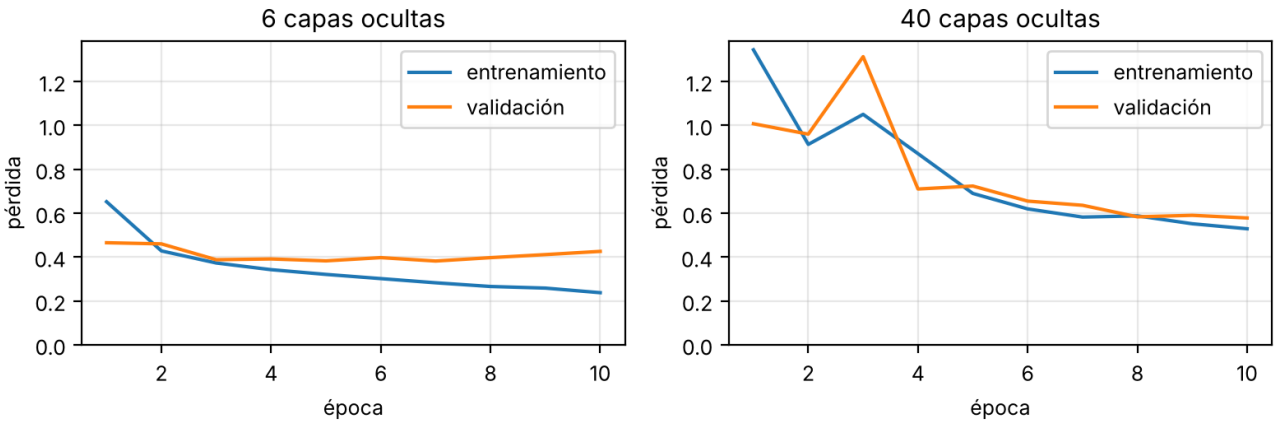
a) ¿Cuál corrida usa lotes de 16? Da dos razones que se vean en la figura o en los tiempos.

La **Corrida Q** usa lotes de 16. Primero, su curva tiene mucho ruido (oscila bruscamente entre 0.1 y 1.5) porque calcula el error con muy pocos datos por paso; segundo, tarda mucho más por época (0.82 s frente a 0.14 s) porque debe hacer 64 veces más actualizaciones de pesos que con lotes de 1024.

b) Si pasas de lotes de 16 a 1024 y quieres avanzar lo mismo por época, ¿qué haces con η? ¿Para qué sirve un *warmup*?

Debes multiplicar η por 64 (aumentarla en la misma proporción en que disminuyen los pasos por época). El *warmup* sirve para subir η gradualmente desde un valor pequeño durante los primeros pasos y evitar que la red diverja al dar pasos enormes cuando sus pesos aún son aleatorios.

6. Profundidad y weight decay



a) Un estudiante dice: «la red de 40 capas sobreajusta; hay que agregarle Dropout». ¿Estás de acuerdo? Justifica con las curvas y propón qué cambiarías.

No estoy de acuerdo: la red de 40 capas no sobreajusta, sino que tiene dificultad para entrenar (subajuste / problema de optimización), pues ambas curvas quedan altas en ~0.55 (con un pico inestable de 1.3 en la época 3) frente al ~0.25 que logra la red de 6 capas. En vez de Dropout, agregaría conexiones residuales (*skip connections*), normalización (BatchNorm / LayerNorm) o ajustaría la inicialización y tasa de aprendizaje.

b) `torch.optim.Adam(..., weight_decay=λ)` frente a `torch.optim.AdamW(..., weight_decay=λ)`: ¿qué hace cada uno con la penalización, y por qué se prefiere AdamW?

Adam mete la penalización dentro del gradiente antes de dividir por la varianza adaptativa, lo que debilita la regularización en los pesos con gradientes históricos grandes. **AdamW** aplica el decaimiento directamente al peso por fuera del paso adaptativo (desacoplado), y se prefiere porque penaliza todos los pesos de manera uniforme y mejora la generalización.